

## File: Makefile

ALL = vector-deadlock vector-global-order vector-try-wait vector-avoid-hold-and-wait vector-nolock

COMMON = vector-header.h main-common.c main-header.h

all: \$(ALL)

clean:

■rm -f \$(ALL) \*~

vector-deadlock: vector-deadlock.c \$(COMMON)

■gcc -o vector-deadlock vector-deadlock.c -Wall -pthread -O

vector-global-order: vector-global-order.c \$(COMMON)

■gcc -o vector-global-order vector-global-order.c -Wall -pthread -O

vector-try-wait: vector-try-wait.c \$(COMMON)

■gcc -o vector-try-wait vector-try-wait.c -Wall -pthread -O

vector-avoid-hold-and-wait: vector-avoid-hold-and-wait.c \$(COMMON)

■gcc -o vector-avoid-hold-and-wait vector-avoid-hold-and-wait.c -Wall -pthread -O

vector-nolock: vector-nolock.c \$(COMMON)

■gcc -o vector-nolock vector-nolock.c -Wall -pthread -O

## File: main-common.c

```
vector_t v[2 * MAX_THREADS];

// used to ensure print outs make sense
pthread_mutex_t print_lock = PTHREAD_MUTEX_INITIALIZER;

void usage(char *prog) {
    fprintf(stderr, "usage: %s [-d (turn on deadlocking behavior)] [-l loops] [-n num_threads] [-t (do timing)]
    exit(1);
}

// only called by one thread (not thread-safe)
void vector_init(vector_t *v, int value) {
    int i;
    for (i = 0; i < VECTOR_SIZE; i++) {
        v->values[i] = value;
    }
    Pthread_mutex_init(&v->lock, NULL);
}

// only called by one thread (not thread-safe)
void vector_print(vector_t *v, char *str) {
    int i;
    for (i = 0; i < VECTOR_SIZE; i++) {
        printf("%s[%d] %d\n", str, i, v->values[i]);
    }
}

void print_info(int call_return, int thread_id, int v0, int v1) {
    if (verbose == 0)
        return;
    Pthread_mutex_lock(&print_lock);
    int j;
    for (j = 0; j < thread_id; j++) printf("%s", " ");
    if (call_return)
        printf("<-add(%d, %d)\n", v0, v1);
    else
        printf("<->add(%d, %d)\n", v0, v1);
    Pthread_mutex_unlock(&print_lock);
}

typedef struct __thread_arg_t {
    int tid;
    int vector_add_order;
    int vector_0;
    int vector_1;
} thread_arg_t;

void *worker(void *arg) {
    thread_arg_t *args = (thread_arg_t *) arg;
    int i, v0, v1;
    for (i = 0; i < loops; i++) {
        if (args->vector_add_order == 0) {
            v0 = args->vector_0;
            v1 = args->vector_1;
        } else {
            v0 = args->vector_1;
            v1 = args->vector_0;
        }

        print_info(0, args->tid, v0, v1);

        vector_add(&v[v0], &v[v1]);

        print_info(1, args->tid, v0, v1);
    }
    return NULL;
}

int main(int argc, char *argv[]) {
```

```

    opterr = 0;
    int c;
    while ((c = getopt (argc, argv, "l:n:vtdp")) != -1) {
■switch (c) {
■case &#x27;l&#x27;;
■    loops = atoi(optarg);
■    break;
■case &#x27;n&#x27;;
■    num_threads = atoi(optarg);
■    break;
■case &#x27;v&#x27;;
■    verbose = 1;
■    break;
■case &#x27;t&#x27;;
■    do_timing = 1;
■    break;
■case &#x27;d&#x27;;
■    cause_deadlock = 1;
■    break;
■case &#x27;p&#x27;;
■    enable_parallelism = 1;
■    break;
■default:
■    usage(argv[0]);
■}
    }

    assert(num_threads &lt; MAX_THREADS);

    pthread_t pid[MAX_THREADS];
    thread_arg_t args[MAX_THREADS];
    int i;
    for (i = 0; i &lt; num_threads; i++) {
■args[i].tid = i;
■if (enable_parallelism == 0) {
■    args[i].vector_0 = 0;
■    args[i].vector_1 = 1;
■} else {
■    args[i].vector_0 = i * 2;
■    args[i].vector_1 = i * 2 + 1;
■}

■if (cause_deadlock && i % 2 == 1)
■    args[i].vector_add_order = 1;
■else
■    args[i].vector_add_order = 0;
    }

    for (i = 0; i &lt; 2 * MAX_THREADS; i++)
■vector_init(&v[i], i);

    double t1 = Time_GetSeconds();

    for (i = 0; i &lt; num_threads; i++)
■pthread_create(&pid[i], NULL, worker, (void *) &args[i]);
    for (i = 0; i &lt; num_threads; i++)
■pthread_join(pid[i], NULL);

    double t2 = Time_GetSeconds();

    fini();
    if (do_timing) {
■printf("Time: %.2f seconds\n", t2 - t1);
    }

    //vector_print(&v[0], "v1");
    //vector_print(&v[1], "v2");
    return 0;
}

```

## File: main-header.h

```
#ifndef __main_header_h__
#define __main_header_h__

#define MAX_THREADS (100)

int loops = 1;
int verbose = 0;
int num_threads = 2;
int do_timing = 0;
int cause_deadlock = 0;
int enable_parallelism = 0;

#endif // __main_header_h__
```

## File: vector-header.h

```
#ifndef __vector_header_h__
#define __vector_header_h__

#define VECTOR_SIZE (100)

typedef struct __vector {
    pthread_mutex_t lock;
    int values[VECTOR_SIZE];
} vector_t;

#endif // __vector_header_h__
```

## File: vector-deadlock.c

```
#include <stdio.h>;
#include <stdlib.h>;
#include <string.h>;
#include <unistd.h>;

#include "mythreads.h";

#include "main-header.h";
#include "vector-header.h";

void vector_add(vector_t *v_dst, vector_t *v_src) {
    Pthread_mutex_lock(&v_dst->lock);
    Pthread_mutex_lock(&v_src->lock);
    int i;
    for (i = 0; i < VECTOR_SIZE; i++) {
        v_dst->values[i] = v_dst->values[i] + v_src->values[i];
    }
    Pthread_mutex_unlock(&v_dst->lock);
    Pthread_mutex_unlock(&v_src->lock);
}

void fini() {}

#include "main-common.c";
```

## File: vector-global-order.c

```
#include <stdio.h>;
#include <stdlib.h>;
#include <string.h>;
#include <unistd.h>;

#include "mythreads.h";

#include "main-header.h";
#include "vector-header.h";

void vector_add(vector_t *v_dst, vector_t *v_src) {
    if (v_dst < v_src) {
        pthread_mutex_lock(&v_dst->lock);
        pthread_mutex_lock(&v_src->lock);
    } else if (v_dst > v_src) {
        pthread_mutex_lock(&v_src->lock);
        pthread_mutex_lock(&v_dst->lock);
    } else {
        // special case: src and dst are the same
        pthread_mutex_lock(&v_src->lock);
    }
    int i;
    for (i = 0; i < VECTOR_SIZE; i++) {
        v_dst->values[i] = v_dst->values[i] + v_src->values[i];
    }
    pthread_mutex_unlock(&v_src->lock);
    if (v_dst != v_src)
        pthread_mutex_unlock(&v_dst->lock);
}

void fini() {}

#include "main-common.c";
```

## File: mythreads.h

```
#ifndef __MYTHREADS_h__
#define __MYTHREADS_h__

#include <pthread.h>;
#include <assert.h>;
#include <stdlib.h>;
#include <sys/time.h>;

void *Malloc(size_t size) {
    void *rc = malloc(size);
    assert(rc != NULL);
    return rc;
}

double Time_GetSeconds() {
    struct timeval t;
    int rc = gettimeofday(&t, NULL);
    assert(rc == 0);
    return (double) ((double)t.tv_sec + (double)t.tv_usec / 1e6);
}

void Pthread_mutex_init(pthread_mutex_t *mutex,
    const pthread_mutexattr_t *attr) {
    int rc = pthread_mutex_init(mutex, attr);
    assert(rc == 0);
}

void Pthread_mutex_lock(pthread_mutex_t *m) {
    int rc = pthread_mutex_lock(m);
    assert(rc == 0);
}

void Pthread_mutex_unlock(pthread_mutex_t *m) {
    int rc = pthread_mutex_unlock(m);
    assert(rc == 0);
}

void Pthread_create(pthread_t *thread, const pthread_attr_t *attr,
    void *(*start_routine)(void*), void *arg) {
    int rc = pthread_create(thread, attr, start_routine, arg);
    assert(rc == 0);
}

void Pthread_join(pthread_t thread, void **value_ptr) {
    int rc = pthread_join(thread, value_ptr);
    assert(rc == 0);
}

#endif // __MYTHREADS_h__
```



## File: vector-nolock.c

```
#include <stdio.h>;
#include <stdlib.h>;
#include <string.h>;
#include <unistd.h>;

#include "mythreads.h";

#include "main-header.h";
#include "vector-header.h";

// taken from https://en.wikipedia.org/wiki/Fetch-and-add
int fetch_and_add(int * variable, int value) {
    asm volatile("<lock; xaddl %%eax, %2;>";
    ■■ :<=> (value)
    ■■ :<a> (value), <m> (*variable)
    ■■ :<memory>;);
    return value;
}

void vector_add(vector_t *v_dst, vector_t *v_src) {
    int i;
    for (i = 0; i < VECTOR_SIZE; i++) {
    ■■ fetch_and_add(&v_dst->values[i], v_src->values[i]);
    }
}

void fini() {}

#include "main-common.c";
```

## File: vector-avoid-hold-and-wait.c

```
#include <stdio.h>;
#include <stdlib.h>;
#include <string.h>;

#include <unistd.h>;

#include "mythreads.h";

#include "main-header.h";
#include "vector-header.h";

// use this to make lock acquisition ATOMIC
pthread_mutex_t global = PTHREAD_MUTEX_INITIALIZER;

void vector_add(vector_t *v_dst, vector_t *v_src) {
    // put GLOBAL lock around all lock acquisition...
    Pthread_mutex_lock(&global);
    Pthread_mutex_lock(&v_dst->lock);
    Pthread_mutex_lock(&v_src->lock);
    Pthread_mutex_unlock(&global);
    int i;
    for (i = 0; i < VECTOR_SIZE; i++) {
        v_dst->values[i] = v_dst->values[i] + v_src->values[i];
    }
    Pthread_mutex_unlock(&v_dst->lock);
    Pthread_mutex_unlock(&v_src->lock);
}

void fini() {}

#include "main-common.c";
```